

Build audit — end-to-end runbook for the IPL power ranking (Strang #5)

- **Slug:** 2026-10-02-ipl-power-ranking-f1
 - **Date:** 2026-10-02
 - **Companion docs:** [specs/2026-10-02-ipl-power-ranking-f1.md](#), [plans/2026-10-02-ipl-power-ranking-f1.md](#), [AGENTS.md](#)
 - **Status:** design validated by a feasibility probe. **No project code written.** The instructor blocker is still open (spec §0) — see Part 3.
-

How to read this document

Every step is marked with who does it:

Mark	Meaning
MANUAL	You do it. An agent cannot — it needs your judgement, your course, or your body.
AI	An agent can do it end to end. You review the output, you do not type it.
BOTH	An agent drafts or executes; you verify against a check you can re-run. Never ship a BOTH step unverified.
AI + HUMAN JUDGEMENT	The agent produces the artefact; the decision is yours and cannot be delegated.

Time estimates are for someone already comfortable with NumPy. They are estimates, not promises.

Part 1 — The feasibility probe, and why it changes the plan

Before writing a plan I ran a throwaway probe (OS temp, deleted, nothing written to the repo) against the raw snapshot. **This was not wasted work — it is stage 7 and stage 8, run early to de-risk.** It found two things that alter the design.

1a. The margin model barely works

Model 1	Massey, least squares on run margins	558 rows × 15 unknowns
	$R^2 = 0.0214$	$ r = 963.4$
	predicts the correct winner 55.0% of the time (a coin flip is 50%)	
Model 1'	win/loss least squares	1218 rows × 15 unknowns
	$R^2 = 0.0071$	predicts the correct winner 54.0%
Model 2	Colley eigenvector	1218 rows
	Perron-Frobenius positivity: confirmed, all entries > 0	

$R^2 = 0.02$ means "difference in team strength = run margin" explains about 2% of the variation in T20 run margins. It is not a ranking model in any useful sense.

1b. The two models do not merely "diverge" — they are unrelated

Spearman(margin-least-squares , Colley)	= 0.000
Spearman(margin-least-squares , win-ls)	= -0.168

Zero rank correlation. And look at what the margin model actually ranks:

team	margin-ls	colley	matches
Kochi Tuskers Kerala	+17.43	0.6%	1 season (2011)
Rising Pune Supergiant	+7.06	1.2%	2 seasons
Gujarat Titans	+6.37	3.3%	5 seasons
Royal Challengers Bengaluru	+4.84	11.2%	19 seasons
...			
Pune Warriors	-5.45	1.9%	3 seasons

team	margin-ls	colley	matches
Rajasthan Royals	-6.08	10.1%	19 seasons
Gujarat Lions	-13.70	1.2%	2 seasons

The margin model is topped by a franchise that **existed for one season** and bottomed by one that existed for two. It is not measuring team strength. It is measuring *how hard short-lived franchises beat or lost badly*.

1c. Why — and this is the real diagnosis

Least squares weights **every match equally**. A team that plays 14 matches and a team that plays 76 are both just "more rows." Kochi Tuskers won almost everything in 2011 and won *large*, so a handful of large-margin wins pins their coefficient far above the pack. Gujarat Lions lost heavily, so their coefficient is extreme negative. Meanwhile the per-match noise in a T20 run margin — a dropped catch, a 19th over, a dew-affected chase — is about **37 runs of standard deviation**, and the entire spread of team strengths is roughly 5–8 runs.

So the signal is buried under roughly 5× its own size in noise, and the few-data teams get the most extreme coefficients.

1d. The three fixes, in order of cost

1. **Frequency-balanced least squares.** Weight each match by $1/(\text{games played by team 1} + \text{games played by team 2})$ so a 14-match franchise cannot dominate. This is the textbook Massey fix and it is **three lines of code**. It directly addresses 1c.
2. **Ridge / shrinkage toward the mean.** Penalise $\|x\|^2$ to pull the noisy short-history coefficients in. This is what Colley's method does naturally via λC .
3. **Report the failure honestly as the finding.** If you do 1 or 2 *and* the margin model still lands near $R^2 = 0.1$, that is a genuine, defensible result: *run margin is not a stable measure of IPL team strength, and win/loss frequency is*.

Do not do any of this quietly. Fitting three models and reporting the best one is cherry-picking. Fit the unbalanced one, show it fail, show the fix, show what it buys. That arc *is* the project.

Part 2 — What the project actually is, restated

The original framing was "build a margin ranking, show it disagrees with the official table." The probe shows something better and more honest:

The obvious approach — treat every IPL match as an equation where the difference in team strength equals the run margin — does not work. Here is the measurement showing it ($R^2 = 0.02$, 55% winner accuracy), here is why (per-match noise swamps team strength, and short-history franchises get extreme coefficients), and here is what fixes it (frequency balancing and shrinkage).

That is a **finding**, not a method demonstration. It also covers all 11 mandated stages honestly, because rank deficiency, projection, QR, eigen and diagonalization are all load-bearing in diagnosing it.

Part 3 — Preconditions: two gates before any code

Step 0.1 — Ask the instructor (**MANUAL — cannot be delegated**)

Spec §0. This gates everything.

Strang problem #5 in the projects book is "*Linear equations + college football team ranking*." May I keep Strang's mathematics and rank **IPL** teams instead, using the mandated 11-stage LA workflow and the Cricsheet dataset? Or must I submit the book's task list on college football data?

- Record the answer in `docs/reasonix/specs/2026-10-02-ipl-power-ranking-f1.md` §0 with who answered, when, and the channel.
- A verbal answer is acceptable **only once written down and confirmed in writing**.

- If the answer is "must submit the book": **stop**. The IPL reframe is illegal and the pick reopens.

Verify: the answer is recorded, not remembered.

Step 0.2 — Baseline commit (**BOTH — agent executes, you decide what is ignored**)

Git exists on branch `main` with **zero commits**, ~1,258 entries staged, and **no .gitignore**. Nothing is recoverable right now.

You decide: whether `ipl_json/` (1,243 files) is committed or ignored. Recommended: **ignore it** — the demo needs only the parsed CSV, and the raw files are re-fetchable. Then provenance (Part 4) is what makes ignoring them safe.

- **Agent:** writes `.gitignore`, stages `.gitignore`, `docs/`, `AGENTS.md`, `CLAUDE.md`, commits.
- **You:** confirm the ignore decision, and confirm `.venv/` is not staged.

Verify: `git log --oneline` shows one commit; `git status --porcelain` does not list `.venv/` after it is created.

Part 4 — Environment and provenance

Step 1.1 — Python environment (**AI**)

```
python -m venv .venv
.venv\Scripts\activate
pip install numpy pandas matplotlib pytest
```

Create `pyproject.toml`, `tests/`, and the directory layout. **The moment `pytest` exists, stage 5 has a real gate.**

Verify: `pytest --version` and `python -c "import numpy, pandas, matplotlib"` both succeed.

Step 1.2 — Provenance record (**BOTH**)

Required by `AGENTS.md` §7 and currently an **outstanding obligation**:

```
data/provenance.json
  source_url, utc_fetch_date, http_status, sha256, row_count, cricsheet_version
```

The snapshot is already on disk at `ipl_json/` — 1,243 files plus `README.txt`.

Compute the SHA-256 of the **downloaded archive** if you still have it; if not, record that the archive itself was not retained and hash the extracted file set instead, saying which you did. **Do not invent a hash.** Record honestly what you hashed.

- **Licence position:** no licence statement for the **match data** exists on `cricsheet.org` (`/licence/` returns 404); the only explicit ODC-BY covers the *Register* dataset. Four third-party sites assert ODC-BY 1.0. **Attribute precisely; do not assert a licence term you did not read from the primary source.** If asked in the viva, say exactly that.

Verify: `provenance.json` parses, `row_count` equals what the parser reports in Step 2.1, and a mismatch fails loudly.

Part 5 — Parse the snapshot

Step 2.1 — Tidy CSV (**AI**) — the highest-risk step in the project

Read all 1,243 files from `ipl_json/` into one CSV.

The traps, all measured, all with their failure mode:

Trap	Correct handling	What happens if you get it wrong
Winner at <code>info.outcome.winner</code>	<code>info["outcome"]</code> <code>["winner"]</code>	<code>info["winner"]</code> raises <code>KeyError: 'winner'</code> in all 1,243 files
Margin at <code>info.outcome.by.runs</code>	<code>.get("by", {})</code> . <code>get("runs")</code>	silent <code>None</code> → 558 rows silently becomes 0
<code>info["season"]</code> is int or str	normalise to <code>str</code> first	<code>len(set(season))</code> returns 24, not 19
<code>outcome.eliminator</code> (16 ties)	no <code>winner</code> key exists → exclude	Super Over matches silently counted as wins
<code>method</code> is at <code>info.outcome.method</code>	full path	<code>info.method</code> is absent in all 1,243 files
Rising Pune Supergiant vs Supergiants	merge as one franchise	invents a 16th unknown with ~1 season of data

Columns: `match_id`, `date`, `season`, `team1`, `team2`, `winner`, `margin_runs`, `margin_wickets`, `method`, `canonical_team1`, `canonical_team2`.

Apply the canonicalisation map from **AGENTS.md §7** — exactly four merges. Do **not** merge Gujarat Lions→Titans or Deccan→Sunrisers; one extra merge takes `n` 15→14 and both take it to 13.

- **MANUAL:** none.
- **Verify:** counts land on **1,243 / 1,218 winner / 558 run-margin / 660 wicket-only / 25 no-winner / 19 raw strings**.

Part 6 — Invariant tests, before anything is built on the data

Step 3.1 — Write the invariant tests (AI)

AGENTS.md §4 makes this mandatory. The numbers below are **measured**, so a red test means the *data or the code* is wrong.

```

assert n_canonical == 15
assert run_margin_rows == 558
assert wicket_only_rows == 660
assert winner_rows == 1218
assert no_winner_rows == 25 # 16 ties + 9 no result
assert total_matches == 1243
assert normalised_season_count == 19

```

Step 3.2 — Structure tests (AI)

```

A = design_matrix(run_margin_rows) # 558 x 15
assert A.shape == (558, 15)
assert np.linalg.matrix_rank(A) == 14 # measured, not inferred
assert A.shape[1] - np.linalg.matrix_rank(A) == 1
assert np.allclose(A.sum(axis=1), 0) # every row sums to zero

```

The rank claim is *predicted* by graph connectivity and *measured* here. Both, never one alone.

Verify: `pytest -q` green. If any assertion is red, stop and fix the parser — everything downstream is meaningless until it is green.

Part 7 — Design matrix

Step 4.1 — Build **A** and **b** (AI)

```

A[i, j] = +1 if team j won match i, -1 if it lost (558 x 15)
b[i]    = signed run margin (+margin if team 1 won,
                             -margin if team 2 won)

```

Rows sum to zero by construction, so the constant vector is in the null space — that is *why* rank is 14, not 15.

Step 4.2 — Build **W** and **C** for Model 2 (AI)

```
W[i,j] = games team i beat team j      (from all 1,218 winner rows)
C[i,j] = games played between i and j
M_colley = W + WT + C
```

Verify: `W.sum()` == 1218, `C` symmetric, `M_colley` entrywise non-negative, and every team has ≥ 1 win and ≥ 1 loss (otherwise Perron-Frobenius gives a reducible matrix and the dominant eigenvector may have zeros).

Part 8 — Mandated stages 2, 3, 4

Step 5.1 — Structure of the space: rank, nullity, null space (AI)

Measured: `rank` = 14, `nullity` = 1, smallest singular value $\approx 4.7e-15$.

The demo line: *"558 matches, 15 teams. The matrix has a null space and it is exactly one-dimensional — add the same number to every team and not one predicted margin changes. So I can only ever rank teams by their **differences**, never absolutely."*

Step 5.2 — Matrix simplification: RREF (AI)

RREF of `A` and of `AT`. The RREF exposes the singular direction explicitly. Note the shape asymmetry: `A` is 558×15 with `rank` 14, so `Row(A)` $\subseteq \mathbb{R}^{15}$ has dimension 14.

Step 5.3 — Remove redundancy (AI + HUMAN JUDGEMENT)

Compute a maximal independent subset of match rows — a basis of $\text{Row}(\mathbf{A})$. This is 14 specific matches out of 558 that determine the ranking; the other 544 are redundant given them.

- **Agent:** computes the subset, reports which matches they are.
- **You:** be able to say *why* a minimal set of 14 matches determines all 15 strengths up to a constant. That is the viva answer, and it is not delegable.

Part 9 — Mandated stages 5, 6

Step 6.1 — Orthogonalization (AI) — two different spaces

Careful, this is the correction in `AGENTS.md` §6:

- **QR of $\mathbf{A} \rightarrow \mathbf{Q}$** is 558×15 with 15 orthonormal columns. Its columns span a 15-dimensional space that **contains** the 14-dimensional $\text{Col}(\mathbf{A})$; the 15th direction is the numerical null direction. It is **not** a basis of $\text{Col}(\mathbf{A})$.
- **QR of $\mathbf{A}^T$** (or Gram–Schmidt on the rows of $\mathbf{A}$) $\rightarrow$ an orthonormal basis of $\text{Row}(\mathbf{A}) \subseteq \mathbb{R}^{15}$, the team space, dimension 14.

Measure, do not assume: check how `numpy.linalg.qr` actually behaves on this rank-deficient $\mathbf{A}$ and determine which leading columns of $\mathbf{Q}$ span $\text{Col}(\mathbf{A})$. This is explicitly deferred to code in the plan.

Step 6.2 — Projection (AI)

The least-squares fit **is** the orthogonal projection of $\mathbf{b}$ onto $\text{Col}(\mathbf{A})$. $\mathbf{r} = \mathbf{b} - \hat{\mathbf{x}}\mathbf{A}$, $\|\mathbf{r}\| = 963.4$ measured.

Verify: $\|\hat{\mathbf{x}} - \mathbf{P} \hat{\mathbf{x}}\| \approx 0$ where $\mathbf{P} = \mathbf{Q}_1 \mathbf{Q}_1^T$ from Step 6.1.

Part 10 — Mandated stages 7, 8

Step 7.1 — Least squares (AI)

Solve $\hat{x} = \operatorname{argmin} \|Ax - b\|$. Three routes, and you should show all three agree:

Route	Note
<code>numpy.linalg.lstsq(A, b)</code>	SVD-based, handles rank deficiency
Normal equations $A^T A x = A^T b$	singular — $A^T A$ is 15×15 rank 14, same null space
Constrained $A^T A x + 11^T x = A^T b$	adding $\Sigma x = 0$ pins the null direction; nonsingular

Correction to my earlier plan: I predicted the normal equations would be ill-conditioned and that any disagreement would be a finding. **That was wrong.**

Measured:

```
cond(A) restricted to its rank    = 5.62           ← benign
cond(A^T A + 11^T)              = 31.55          ← benign
constrained solve vs lstsq      = 4.8e-14 max |diff|
```

The normal equations are numerically fine here. Show that they agree — it is a clean result and it means the constrained route is the right one to demonstrate, since it makes the null space explicit rather than hiding it.

Normalise x — only differences are identifiable, so subtract the mean.

Step 7.2 — Colley eigenvector (AI)

Power iteration on `M_colley`; normalise so $\Sigma r = 1$. Measured: **all entries strictly positive**, confirming Perron–Frobenius rather than assuming it. Top two eigenvalues are real and distinct.

Step 7.3 — The comparison (AI + HUMAN JUDGEMENT)

```
Spearman( margin-ls , Colley ) = 0.000
```

The two models rank the teams in **completely unrelated orders**. That is the result, and explaining *why* is the intellectual work you own.

Part 11 — Mandated stage 9

Step 8.1 — Diagonalization (AI)

$A^T A$ is symmetric positive semi-definite by construction (measured: exactly symmetric, minimum eigenvalue $\approx -9.2e-15$, i.e. zero within float error). So the spectral theorem applies directly — real non-negative eigenvalues, orthogonal diagonalisation. No contrivance needed.

The free connection: $A^T A = V \Sigma^2 V^T$, so the eigendecomposition of $A^T A$ is the SVD of A . One line, and it links stage 9 to Strang [#14](#) without attempting the held stretch variant.

Verify: all eigenvalues of $A^T A$ are $\geq -1e-10$; $V^T V = I$ to tolerance; $A \approx U \Sigma V^T$ reconstructs.

Part 12 — Mandated stage 10, and the finding

Step 9.1 — Rankings and confidence intervals (AI)

Per-team standard errors from the constrained covariance:

$$\text{Cov}(\hat{x}) = \sigma^2 (A^T A + 11^T)^{-1}$$

$$\sigma^2 = \|r\|^2 / (m - \text{rank}) = \|r\|^2 / 544$$

This turns an unjustified ranking into an honest one, and it is the direct answer to "which team do you trust least?"

Step 9.2 — The official points table (AI + HUMAN JUDGEMENT)

You can compute the official IPL table from the snapshot — do not scrape it:

```
win = 2 points, loss = 0, tie = 1, no-result = 1
```

Be explicit about which table you compare against — per-season or all-time. Say which, do not leave it ambiguous.

Optional stretch: Net Run Rate needs runs scored and overs faced from the innings data. It breaks ties at the qualification cutoff often enough that omitting it is a visible gap. If you compute it, state that you did; if not, say why not.

Step 9.3 — Figures (AI)

1. Both rankings side by side with the official table
2. Divergence plot
3. **The diagnostic figure that carries the finding:** fitted vs actual margin, or residual vs predicted magnitude — showing the leverage of the short-history franchises
4. Per-team $x \pm se$

Verify: every figure regenerates deterministically from the committed CSV, offline.

Part 13 — The diagnosis, which is the actual contribution

Step 10.1 — Frequency-balanced least

squares (AI + HUMAN JUDGEMENT)

Weight match i by $w_i = 1/(\text{games}(t1) + \text{games}(t2))$ and solve the weighted normal equations. Kochi Tuskers and Gujarat Lions should stop dominating.

- **Agent:** implements it, reports R^2 and the new ranking.
- **You:** predict what happens **before** you run it, then check. A prediction you were right about is a viva line; a number you only got by running code is not.

Step 10.2 — Report the arc honestly (MANUAL)

The full story, in order: unbalanced model $\rightarrow R^2 = 0.02 \rightarrow$ diagnosis (noise vs signal; few-data leverage) $\rightarrow$ frequency balancing $\rightarrow$ what it buys $\rightarrow$ conclusion about whether run margin can rank IPL teams at all.

MANUAL, and it is the part that earns the 5 viva marks. An agent can write the sentences. It cannot decide what the honest conclusion is when the fix buys less than expected.

Part 14 — Demo, report, viva

Step 11.1 — Offline demo hardening (AI)

Must run with the network off, from a clean clone, in ~10 seconds, twice with identical output. No absolute paths, no machine-specific env vars.

Step 11.2 — Report (BOTH)

Agent drafts; **you** own every claim in it. Read [AGENTS.md](#) §4 before accepting a sentence.

Step 11.3 — Viva script (BOTH — agent drafts, you memorise)

Question	Answer
Why least squares, not exact solving?	558 equations, 15 unknowns. Over-determined. (Not 1,218 — that is Model 2's row count.)
Your matrix is singular. Problem?	No. Nullity 1 = the constant vector. Only <i>differences</i> are identifiable. That is the insight.
Why only 558 matches?	Run margin is the only signed numeric margin. The other 660 give wickets, a different scale. I refused to invent a conversion.
Why doesn't the eigen method match your least squares?	Different assumption, different data — Massey uses margin on 558 rows, Colley uses wins on 1,218.
Why is your ranking worse than a coin flip?	This is the question your project answers. $R^2 = 0.02$, 55% winner accuracy. Per-match noise ($\sigma \approx 37$ runs) swamps the signal (spread $\approx 5\text{--}8$ runs), and least squares weights every match equally, so short-history franchises get extreme coefficients.
How do you know your ranking is better than the official one?	Honest answer: it isn't provably better, it is differently informed — and I measured that mine is worse, which is why the official table wins on predictive accuracy. Say exactly this.
Which team's estimate do you trust least?	The standard errors tell you; name one with its se .

Part 15 — Where to spend the hours

Step	Stage	Who	Est.	Gate
0.1 Instructor question	—	MANUAL	10 min	blocks everything

Step	Stage	Who	Est.	Gate
0.2 Baseline commit	—	BOTH	10 min	<code>git log</code> non-empty
1.1 Environment	—	AI	15 min	<code>pytest --version</code>
1.2 Provenance	—	BOTH	30 min	<code>row_count</code> matches
2.1 Parse	—	AI	60–90 min	counts match table
3.1–3.2 Invariant tests	—	AI	45 min	<code>pytest green</code>
4.1–4.2 Design matrices	1	AI	45 min	<code>A.sum(axis=1) == 0</code>
5.1–5.3 Structure	2,3,4	AI + HUMAN	90 min	rank/nullity measured
6.1–6.2 Orthogonalization	5,6	AI	90 min	projection identity
7.1–7.3 Least squares + eigen	7,8	AI + HUMAN	2 h	three routes agree
8.1 Diagonalization	9	AI	45 min	eigs ≥ 0 , $V^T V = I$
9.1–9.3 Output + SE	10	AI + HUMAN	2 h	deterministic
10.1–10.2 Diagnosis	—	AI + HUMAN	2 h	the contribution
11.1–11.3 Demo, report, viva	—	BOTH	3 h	offline, reproducible
Total			≈ 14 h	

The feasibility probe already covers 4.1, 7.1–7.3 and part of 9.1. Realistically **≈ 11 h remains**, and several hours of that are judgement, not typing.

Part 16 — What is still unverified

Recorded so the next session does not mistake these for settled:

- **The blocker.** Open. Nothing above is authorised project code until it is answered.

- **The frequency-balanced and ridge variants have not been fitted.** Part 13 predicts they will help. That is a prediction, not a measurement.
- **`numpy.linalg.qr` behaviour on this rank-deficient `A`.** Deferred to code in Step 6.1.
- **NRR** is not implemented. The official-table comparison currently rests on win/loss only.
- **The `si/s[-2]` condition number of 5.62** excludes the null direction by construction; it says the matrix is well-conditioned *on its range*, not that `A` is invertible. `A` is not invertible and cannot be.
- **Licence** remains unverified from the primary source.